

Jitsu QA – Web App Release Process (Full Guide)

JITSU QA TEAM

Web App Release Process

Full Guide for QA

New isolated-release-branch SDLC – what changed and what QA needs to do

📄 INSIDE THIS DOC

- ✓ What changed in the new release process
- ✓ Edge cases now covered (hotfix, partial, multi-team, etc.)
- ✓ QA workflow on each environment
- ✓ Where to ask for help (4 channels in priority order)
- ✓ Reference docs in `gojitsucom/jt-sdlc-docs`

Effective: This sprint (now live for all APIs, Workers, Common Libraries, Web Apps)

Version 1 – May 2026

Table of Contents

1.	Overview – What Changed
2.	Edge Cases Now Covered
3.	QA Workflow on Each Environment
4.	Reference Documentation
5.	Where to Ask for Help
6.	QA Action Checklist

1. Overview – What Changed

As of this sprint, every API, Worker, Common Library, and Web App at Jitsu is now on the new isolated-release-branch release process. For your next release on any repo, the team must follow the new SDLC.

Key change

Each release is now isolated in its own release branch – separate from ongoing development work. This gives the team:

- Predictable release scope – you know exactly what's in each release.
- Cleaner separation between hotfixes and feature releases.
- Better multi-team coordination – each team's release is isolated.
- Clearer gate points between Staging, Beta, and Production.

Why QA should care

- Test scope is now more predictable – you know what's in each release branch.

- Hotfix and feature releases follow different paths – different test scopes apply.
- Multi-team work on Staging no longer blocks your testing because each release branch is isolated.
- Verification comments on Jira should mention the branch and environment for traceability.

📊 Environment Promotion Flow (visual)

Figure 1 – Each environment has its own QA gate. Promotion happens only after QA verification.

2. Edge Cases Now Covered

The new SDLC docs have been refreshed based on observations from previous sprint cycles. The following edge cases – which previously required ping-pong DevOps – are now documented:

📊 Branch Flow – Standard Release vs Hotfix

Figure 2 – Standard release (top) goes dev → release/ → main. Hotfix (bottom) skips dev and branches directly from main.*

Edge case	**What it means for QA**
--- ---	
Mid-sprint partial releases A subset of features released before sprint end. Test scope is hedged – only verify features in the partial release, not other in-progress work. Confirm with PM/Lead what's IN scope.	
Hotfix flows Urgent prod fixes have a dedicated flow. QA test scope = the fix + nearby regression. Don't run full E2E – speed matters.	
Dependency pinning timing When common libraries are pinned to specific versions during a release. QA may see version-locked dependencies in build notes – refer to docs if unclear.	
Conflict resolution on promote/* branches When conflicts arise during code promotion, dev resolves them on the promote branch. QA must re-verify after resolution – the resolved code may differ from original test.	
Multi-team Staging coordination Multiple teams may deploy to Staging concurrently. Announce your test sessions in #qa-web, use Test_ prefix on data, cleanup after.	
💡 The guides are dense – treat as reference DevOps explicitly noted: "The guides are dense – we don't expect you to read them end-to-end in one sitting. Treat them as a reference. Read once enough to recognize the new commands and gates; come back to specific sections when you need them."	

3. QA Workflow on Each Environment

This section describes what QA should expect and do at each gate of the new release process.

3.1 Staging Phase

What arrives here

Continuous integration of new features as they merge into the integration branch. Multiple teams' features may be on Staging simultaneously.

QA actions

- Dev signals "ready for QA" on Jira ticket.

- QA pulls the ticket, reviews AC, prepares test cases.
- Test the feature on Staging URL (e.g. <https://dashboard.staging.gojitsu.com>).
- Watch Datadog logs in parallel – catch backend errors not visible in UI.
- Smoke test tickets released BEFORE yours on the same repo (see note below) – confirms previous merges are intact.
- If PASS → comment on Jira: "Tested on Staging – PASSED" + build version + test case link.
- If FAIL → log bug per Bug Reporting template (Training Doc Section 8 – Step 8).

****⚠ Staging stability expectations**** Staging may be unstable. Multiple teams' features land here concurrently. If you see an unrelated error during testing, check Datadog timestamp before logging a bug – it might be another team's issue. Cross-check in #qa-web before opening a ticket.

****🔄 Smoke test previously-released tickets on the SAME repo**** Before completing your ticket on Staging, smoke test the tickets that were released BEFORE yours on the same repo. This ensures the previous developer's code was actually merged into the codebase you're testing against – and that your ticket's deploy didn't accidentally regress earlier work. Why: when multiple devs commit to the same repo, code from earlier tickets must still be present and working in the current Staging build. A missing merge or revert can silently break already-released features. How: • Check the recent merge history of the repo (ask dev or look at PRs). • Identify tickets released to Staging before yours on the same repo. • Run a quick smoke test on those tickets' happy paths. • If something is broken → cross-check with the original ticket's QA before logging a regression.

****3.2 Beta Phase****

****What arrives here****

Code from the release branch – a frozen, isolated snapshot of features intended for production. Tagged as a release candidate.

****QA actions****

- Check what's available on Beta – not all components/workers are deployed (see note below).
- Re-verify features on Beta – same test cases as Staging.
- Verify Beta config matches Production (data flags, feature flags, env vars).
- Smoke test tickets released BEFORE yours on the same repo – ensures prior merges are intact.
- Run smoke regression of related features (Beta is the last gate before Prod).
- Comment on Jira: "Verified on Beta – PASSED" + build version + scope tested + evidence.
- If FAIL → bug logged for the release branch. Dev applies fix directly on the release branch.

| ****⚠ Beta limitations – not all components are available**** The Beta environment does NOT have all components/services deployed. In particular: •

Workers may not be available on Beta • Some web apps may not have a Beta deployment • Some integrations may not run Before testing on Beta, check whether your ticket's component (worker or web app) actually has a Beta deployment. If it doesn't exist on Beta, you can SKIP Beta verification – but note that clearly in the Jira comment (e.g. "Beta verification skipped – no Beta deployment for [component]"). Move directly to Production verification after Staging passes. |

| --- |

| **⚠️ Re-verify after any new commit on release branch** If dev applies any fix to the release branch, a new build is generated with a new commit hash. The new build = different code = you must re-verify. Don't assume "close enough" – verify the same test cases on the new build. |

| --- |

3.3 Production Phase

What arrives here

The release branch merges into the production branch with a strict version tag. The image is immutable – same code as approved on Beta.

QA actions

- Smoke check key user flows of the feature (do NOT run full regression on Prod).
- Smoke test tickets released BEFORE yours on the same repo – ensures prior merges are intact in Prod.
- Follow the 8 Production Rules (Training Doc Section 10).
- Cleanup any test data created during Prod verification.
- Comment on Jira: "Verified on Production – PASSED" + production version + evidence.
- Move ticket to Done / Released.

| **🔥 Production smoke check – keep it MINIMAL** Production is live. Smoke check only the primary user-facing flow of the feature. Do NOT run full test cycles on Production unless explicitly required. Always follow the 8 Production Rules (e.g. Test_ prefix, cleanup after testing, Jitsu email only). |

| --- |

3.4 Hotfix Path (Exception)

When this triggers

Critical production issue that can't wait for the standard cycle. Hotfix bypasses the normal Staging → Beta path.

QA actions

- Dev notifies #qa-web with urgency.
- Verify the fix in a pre-prod environment as designated by dev.
- Test scope: FIX itself + immediate surrounding feature (regression nearby).
- Do NOT do full app regression – speed matters.
- If PASS → dev deploys hotfix to Production.

- QA smoke check on Prod after deploy.
- Comment on Jira with hotfix version + scope tested.

💡 Hotfix testing – speed vs safety Hotfix testing balances two conflicting goals: fast (because Prod is broken) AND safe (no regression). Solution: test the fix thoroughly + smoke check 2-3 related flows. If hotfix touches 1 endpoint, test that endpoint + its caller(s). Don't run full E2E.

4. Reference Documentation

Authoritative source: the [gojitsucom/jt-sdlc-docs](https://github.com/gojitsucom/jt-sdlc-docs) repo on GitHub. These docs are the official source of truth for the new SDLC.

Doc	**When to read**
APIs / Workers – Full Guide	[NEW-code-to-production-guide.md](#) – Deep reference for full release process on APIs and workers. Read sections as needed.
APIs / Workers – Cheatsheet	[NEW-code-to-prod-Quick-Reference-Cheatsheet.md](#) – Quick reference for common commands and gates. Bookmark this.
Common Libraries – Full Guide	[NEW-common-libs-sdlc.md](#) – For work on shared libraries between services.
Common Libraries – Cheatsheet	[NEW-common-libs-Quick-Reference-Cheatsheet.md](#) – Quick reference for common library release.

💡 How to use as a QA You don't need to read these end-to-end. Use them as reference when you need to: • Understand a command in a PR description • Figure out why a release branch was created in a certain way • Look up a gate or convention dev mentioned • Verify what's expected at each environment transition

5. Where to Ask for Help

Use these resources in order – most questions can be answered without pinging anyone.

#	**Resource**	**When to use**
1	**Augment / Claude Code in dev session**	**RECOMMENDED.** Add [gojitsucom/jt-sdlc-docs](https://github.com/gojitsucom/jt-sdlc-docs) as context, ask step-by-step as you work. Best for hands-on tasks.
2	**DevOps GPT (ChatGPT)**	Pre-trained on the SDLC. Fastest for "how do I...", "what does this workflow do", "I see this CI error".
3	**Web chat with Claude + GitHub connector**	Open claude.ai/new, attach [gojitsucom/jt-sdlc-docs](https://github.com/gojitsucom/jt-sdlc-docs) via GitHub connector, ask anything.
4	**Ping @devops in #devops-support**	Last resort. After trying 1-3. Mention what you tried and why it didn't resolve.

💡 One-time setup for option 1 (Claude Code) 1. Authenticate GitHub CLI: `gh auth login` 2. Open a Claude Code session at the root where Jitsu repos are cloned (centralized) or inside a specific repo 3. Drop the `CLAUDE.md` template from [gojitsucom/jt-sdlc-docs](https://github.com/gojitsucom/jt-sdlc-docs) into that directory The agent will then walk you through SDLC tasks step-by-step. Will be later migrated as skills/commands to deshi.

6. QA Action Checklist

Use this checklist when verifying a ticket under the new release process.

📊 QA Decision Flow – Which Test Scope?

Figure 3 – Use this flow to decide test scope: hotfix (fast, narrow), partial (only scope), or standard (full cycle).

📋 Before testing

- Confirm which release this ticket is in (standard or partial?).
- Confirm test environment (Staging / Beta / Production).
- Know the build version of the deployment you're testing.
- If on Beta – verify config matches Production.

📋 During testing

- Watch Datadog logs in parallel.
- Test according to AC + test cases.
- Note any unrelated issues to cross-check with other teams later.

📋 If testing on Beta – extra steps

- Run smoke regression on related features (Beta is the last gate).
- Verify config flags match Production.
- If dev re-commits to release branch → RE-VERIFY with new build.

📋 If hotfix

- Test the fix only + nearby regression.
- Do NOT run full E2E.
- Smoke check on Prod after deploy.

📋 After testing

- After testing on EACH environment (Staging, Beta, Prod) – comment on Jira with status + evidence.
- Status must be clear: "Verified on [Environment] – PASSED" (or FAILED).
- Always attach evidence (screenshots/video) below the ticket to prove you tested.
- For mobile tickets – include the version you verified on.
- Move ticket to next status.
- If multi-team Staging – post completion update in #qa-web.
- Cleanup test data on Production.

📋 Verification Comment Templates

Web ticket – minimum required: status + environment + evidence. Build version + scope are recommended for traceability.

✅ Minimum example (Web)

Verified on Staging – PASSED

[Attach evidence: screenshots / video]

✅ Full example with traceability (Web)

Verified on Beta – PASSED

Build: [version + commit hash, e.g. 0.15.0-rc-14a2e3d]

Test cases: [QMetric link]

Scope verified:

- [Scenario 1]
- [Scenario 2]
- Smoke test of previous tickets on same repo: OK

Evidence:

[Attach screenshots / video]

✅ Beta SKIPPED example (when no Beta deployment)

Beta verification SKIPPED

Reason: [Component name] has no Beta deployment (worker / app not on Beta).

Proceeding to Production verification after Staging passes.

📱 Mobile example – MUST include verified version

Verified on Staging – PASSED

App: Driver App (Jitsu Drive)

Version verified: 2.8.2 (Firebase Staging build)

[Attach evidence: screenshots / video]

Verified on Production – PASSED

App: Driver App (Jitsu Drive)

Version verified: 2.9.0 (TestFlight)

[Attach evidence]

🚨 Evidence is MANDATORY on every verification comment Every Jira verification comment (Staging, Beta, Prod) MUST have evidence attached below the comment – screenshots or a screen recording proving you actually tested. "Verified – PASSED" without evidence is not acceptable: it leaves no proof of testing and the ticket may be sent back.

End of Web App Release Process – Full Guide 🚀

For day-to-day quick reference, see the Cheatsheet companion doc.

Page of